

EXPERIMENT - 39

1. Problem Statement

Part 1. Write a C program containing a variadic function `double weightedAverage(int count, ...)`. The function should accept "count" and pairs of values, each pair consisting of a value and a weight, and should calculate and return the weighted average. For example `weightedAverage(3, 80.0, 0.3, 75.0, 0.2, 90.0, 0.5)` has `count = 3`, so the three pairs are (80.0, 0.3), (75.0, 0.2) and (90.0, 0.5), and the answer is $(80 \times 0.3 + 75 \times 0.2 + 90 \times 0.5) / (0.3 + 0.2 + 0.5) = 84.00$. The program must validate that all weights are non-negative, handle the case where the total weight is zero, display an appropriate error message for invalid input, and use `stdarg.h` correctly.

Part 2. Write a C program that accepts any number of command-line arguments representing integers. The program should validate each argument and ignore non-integer values, process the valid integers and display them in ascending order, display the minimum, maximum, average, median and second-largest distinct value, and use appropriate user-defined functions for validation, sorting and calculation. The source code is assumed to be named `program.c` and compiled as `$cc program.c -o program`.

2. Algorithm

Part 1, Function `weightedAverage()`

Step 1: If `count` is less than or equal to 0, display an error message and return 0.

Step 2: Set `total_product = 0` and `total_weight = 0`, and call `va_start` on `count`.

Step 3: Repeat for `i = 0` to `count - 1`:

 Read value and then weight using `va_arg` with the type `double`.

 If weight is less than 0, display an error message, call `va_end` and return 0.

 Add `value × weight` to `total_product` and add weight to `total_weight`.

Step 4: Call `va_end`.

Step 5: If `total_weight` is 0, display an error message and return 0.

Step 6: Return `total_product` divided by `total_weight`.

Part 1, Main Program

Step 1: START

Step 2: Call `weightedAverage()` with the pairs given in the question and display the value it returns.

Step 3: Repeat Step 2 for a second set of pairs, for a set whose weights are all zero, for a set containing a negative weight, and for a count of 0.

Step 4: STOP

Part 2, Main Program

Step 1: START

Step 2: Set `n = 0` and repeat for `i = 1` to `argc - 1`:

 If `isInteger(argv[i])` is 1, store `toInteger(argv[i])` in `a[n]` and add 1 to `n`.

 Otherwise display that the argument was ignored.

Step 3: If n is 0, display that no valid integers were given and stop.

Step 4: Call `sortArray()` and display $a[0]$ to $a[n - 1]$.

Step 5: Display $a[0]$ as the minimum and $a[n - 1]$ as the maximum.

Step 6: Display `average(a, n)` and `median(a, n)`.

Step 7: If $a[0]$ is not equal to $a[n - 1]$, display `secondLargest(a, n)`, otherwise display that there is no second distinct value.

Step 8: STOP

Part 2, Function `isInteger()`

Step 1: Set $i = 0$, and if $s[0]$ is '-' or '+', set $i = 1$.

Step 2: If $s[i]$ is '\0', return 0.

Step 3: Repeat while $s[i]$ is not '\0': if $s[i]$ is not between '0' and '9', return 0.

Step 4: Return 1.

Part 2, Function `toInteger()`

Step 1: Set $value = 0$, $sign = 1$ and $i = 0$.

Step 2: If $s[0]$ is '-', set $sign = -1$ and $i = 1$. If $s[0]$ is '+', set $i = 1$.

Step 3: Repeat while $s[i]$ is not '\0': set $value = value \times 10 + (s[i] - '0')$.

Step 4: Return $sign \times value$.

Part 2, Function `sortArray()`

Step 1: Repeat for $i = 0$ to $n - 2$:

Set $small = i$.

Repeat for $j = i + 1$ to $n - 1$: if $a[j]$ is less than $a[small]$, set $small = j$.

Swap $a[i]$ and $a[small]$.

Step 2: Return to the calling function.

Part 2, Functions `average()`, `median()` and `secondLargest()`

Step 1: `average()` adds all the elements into a double sum and returns sum divided by n .

Step 2: `median()` returns $a[n / 2]$ if n is odd, and the average of $a[n / 2 - 1]$ and $a[n / 2]$ if n is even.

Step 3: `secondLargest()` walks from $i = n - 2$ down to 0 and returns the first $a[i]$ that is not equal to $a[n - 1]$.

3. Test Case Table

#	Input	Expected Output	Actual Output	Result
1	weightedAverage(3, 80.0, 0.3, 75.0, 0.2, 90.0, 0.5)	84.00, the example from the question	Weighted average: 84.00	PASS
2	weightedAverage(2, 60.0, 1.0, 90.0, 3.0)	$330 / 4 = 82.50$	82.50, so weights do not have to add up to 1	PASS
3	weightedAverage(2, 70.0, 0.0, 95.0, 0.0)	An error for the total weight being zero, and no division	Said the total weight is zero	PASS
4	weightedAverage(3, 80.0, 0.5, 75.0, -0.2, 90.0, 0.5)	An error naming the negative weight	Named -0.20 and stopped	PASS
5	weightedAverage(0)	An error, since there are no pairs at all	Said the count must be at least 1	PASS
6	./program 12 5 abc 8 12 20 3	abc ignored, 3 5 8 12 12 20, min 3, max 20, average 10.00, median 10.00, second-largest 12	Matches the question line for line	PASS
7	./program 7 -3 xyz 15 7 2 99	-3 2 7 7 15 99, average $127 / 6 = 21.17$, median $(7 + 7) / 2 = 7.00$	21.17 and 7.00, so a negative number sorts correctly	PASS
8	./program 42 abc 7.5 -8 42	Both abc and 7.5 ignored, second-largest -8	7.5 rejected as well, second-largest -8	PASS
9	./program 5 5 5	No second distinct value exists	Said there is none instead of printing a wrong number	PASS
10	./program hello world	Both ignored, and nothing is calculated	No valid integers were given	PASS

4. Source Code

weighted.c

```
/* Experiment 39, part 1. The function is given a count first and then
   that many value and weight pairs, and hands back their weighted
   average. The "..." on its own does nothing useful, stdarg.h is what
   gives you a way to actually reach those arguments. */

#include <stdio.h>
#include <stdarg.h>

double weightedAverage(int count, ...);

int main(void)
{
    double avg;

    printf("Call 1: weightedAverage(3, 80.0, 0.3, 75.0, 0.2, 90.0, 0.5)\n");
    avg = weightedAverage(3, 80.0, 0.3, 75.0, 0.2, 90.0, 0.5);
    printf("Weighted average: %.2f\n\n", avg);

    printf("Call 2: weightedAverage(2, 60.0, 1.0, 90.0, 3.0)\n");
    avg = weightedAverage(2, 60.0, 1.0, 90.0, 3.0);
    printf("Weighted average: %.2f\n\n", avg);

    printf("Call 3: weightedAverage(2, 70.0, 0.0, 95.0, 0.0)\n");
    avg = weightedAverage(2, 70.0, 0.0, 95.0, 0.0);
    printf("Weighted average: %.2f\n\n", avg);

    printf("Call 4: weightedAverage(3, 80.0, 0.5, 75.0, -0.2, 90.0, 0.5)\n");
    avg = weightedAverage(3, 80.0, 0.5, 75.0, -0.2, 90.0, 0.5);
    printf("Weighted average: %.2f\n\n", avg);

    printf("Call 5: weightedAverage(0)\n");
    avg = weightedAverage(0);
    printf("Weighted average: %.2f\n", avg);

    return 0;
}

/* count is the last named parameter, which is why va_start is given
   that name. After that va_arg pulls the arguments off one at a time,
   in the order they were written, so each turn of the loop takes two
   of them, the value and then its weight.
   The type inside va_arg must be double, not float. I had float there
   at first, because weights like 0.3 looked small to me, and the
   program printed 0.00 and -nan. Anything of type float that goes into
   a "..." function is turned into a double before it arrives, so
   asking for a float reads only half of one and the rest is garbage. */
double weightedAverage(int count, ...)
{
    va_list args;
    int i;
    double value, weight;
    double total_product = 0.0, total_weight = 0.0;

    if (count <= 0) {
        printf("Error: the count must be at least 1\n");
        return 0.0;
    }

    va_start(args, count);

    for (i = 0; i < count; i++) {
        value = va_arg(args, double);
        weight = va_arg(args, double);

        if (weight < 0) {
            printf("Error: the weight %.2f is negative\n", weight);
            va_end(args);
            return 0.0;
        }

        total_product += value * weight;
        total_weight += weight;
    }

    if (total_weight == 0.0)
        return 0.0;

    return total_product / total_weight;
}
```

```

        return 0.0;
    }

    total_product = total_product + value * weight;
    total_weight = total_weight + weight;
}

va_end(args);

/* This cannot go inside the loop. Every weight can be zero without
   a single one of them being negative, so the loop sees nothing
   wrong and only the total at the end shows the problem. Dividing
   by it would have given inf. */
if (total_weight == 0.0) {
    printf("Error: the total weight is zero\n");
    return 0.0;
}

return total_product / total_weight;
}

```

program.c

```
/* Experiment 39, part 2. The numbers are typed after the program name
   on the command line instead of being read with scanf, which is why
   main takes argc and argv this time. Anything that is not a whole
   number gets reported and dropped, and whatever survives is sorted
   and then measured. */

#include <stdio.h>

int isInteger(char s[]);
int toInteger(char s[]);
void sortArray(int a[], int n);
double average(int a[], int n);
double median(int a[], int n);
int secondLargest(int a[], int n);

int main(int argc, char *argv[])
{
    int a[100];
    int n = 0, i;

    /* i starts from 1, not 0. argv[0] is the name of the program
       itself, and my first run cheerfully printed "Invalid argument
       ignored: ./program" before getting to the real ones. */
    for (i = 1; i < argc; i++) {
        if (isInteger(argv[i]))
            a[n++] = toInteger(argv[i]);
        else
            printf("Invalid argument ignored: %s\n", argv[i]);
    }

    if (n == 0) {
        printf("No valid integers were given\n");
        return 0;
    }

    sortArray(a, n);

    printf("Valid integers in ascending order:");
    for (i = 0; i < n; i++)
        printf(" %d", a[i]);
    printf("\n");

    printf("Minimum: %d\n", a[0]);
    printf("Maximum: %d\n", a[n - 1]);
    printf("Average: %.2f\n", average(a, n));
    printf("Median: %.2f\n", median(a, n));

    /* The list is sorted by now, so if the smallest and the largest
       are the same number then every value in between is that number
       too, and there is no second distinct value to look for. */
    if (a[0] != a[n - 1])
        printf("Second-largest distinct value: %d\n", secondLargest(a, n));
    else
        printf("Second-largest distinct value: none, every value is the same\n");

    return 0;
}

/* A sign is allowed but only at the front, and something has to come
   after it, otherwise a lone "-" would pass as a number. Everything
   from there on has to be a digit, which is how 7.5 gets thrown out,
   since the dot is not one. */
int isInteger(char s[])
{
    int i = 0;

    if (s[0] == '-' || s[0] == '+')
        i = 1;

    if (s[i] == '\0')
        return 0;
```

```

    for (; s[i] != '\0'; i++)
        if (s[i] < '0' || s[i] > '9')
            return 0;

    return 1;
}

/* The character '5' is not the number 5, it is character 53, so
   subtracting '0' from a digit leaves its value behind. Every new
   digit pushes the earlier ones one place to the left, which is what
   the times 10 is doing, and the sign goes back on at the end. */
int toInteger(char s[])
{
    int i = 0, value = 0, sign = 1;

    if (s[0] == '-') {
        sign = -1;
        i = 1;
    } else if (s[0] == '+') {
        i = 1;
    }

    for (; s[i] != '\0'; i++)
        value = value * 10 + (s[i] - '0');

    return sign * value;
}

/* Selection sort, because it was the easiest one for me to get right.
   Each round looks through whatever is still unsorted, finds the
   smallest, and swaps it to the front of that part, so one more
   position is settled every time. */
void sortArray(int a[], int n)
{
    int i, j, small, temp;

    for (i = 0; i < n - 1; i++) {
        small = i;
        for (j = i + 1; j < n; j++)
            if (a[j] < a[small])
                small = j;

        temp = a[i];
        a[i] = a[small];
        a[small] = temp;
    }
}

/* sum is a double on purpose. If it were an int then an int divided
   by an int would drop everything after the decimal point, and 127 / 6
   would have come out as 21 instead of 21.17. */
double average(int a[], int n)
{
    int i;
    double sum = 0.0;

    for (i = 0; i < n; i++)
        sum = sum + a[i];

    return sum / n;
}

/* This is only called after sorting, otherwise the middle of the array
   means nothing. An odd count has one element sitting in the middle,
   an even count has two, so those two get averaged. The 2.0 keeps the
   division in decimals. */
double median(int a[], int n)
{
    if (n % 2 == 1)
        return a[n / 2];

    return (a[n / 2 - 1] + a[n / 2]) / 2.0;
}

```

```
/* Starting at n - 2 steps over the largest value itself, and then the
   loop keeps walking back past any repeats of it until it meets a
   different number. That is why 3 5 8 12 12 20 gives 12 and not 20,
   and why a second copy of the maximum would not fool it either. */
int secondLargest(int a[], int n)
{
    int i;

    for (i = n - 2; i >= 0; i--)
        if (a[i] != a[n - 1])
            return a[i];

    return a[n - 1];
}
```

5. Compilation and Output

I compiled both with `gcc -Wall -Wextra`, since those flags bring out the small warnings as well, and neither of them complained about anything.

Part 1, the Variadic Function

```
adilshakil — -zsh — 124x31
adilshakil@ADILs-MacBook-Air MyAssignment % gcc -Wall -Wextra weighted.c -o weighted
adilshakil@ADILs-MacBook-Air MyAssignment % ./weighted
Call 1: weightedAverage(3, 80.0, 0.3, 75.0, 0.2, 90.0, 0.5)
Weighted average: 84.00

Call 2: weightedAverage(2, 60.0, 1.0, 90.0, 3.0)
Weighted average: 82.50

Call 3: weightedAverage(2, 70.0, 0.0, 95.0, 0.0)
Error: the total weight is zero
Weighted average: 0.00

Call 4: weightedAverage(3, 80.0, 0.5, 75.0, -0.2, 90.0, 0.5)
Error: the weight -0.20 is negative
Weighted average: 0.00

Call 5: weightedAverage(0)
Error: the count must be at least 1
Weighted average: 0.00
adilshakil@ADILs-MacBook-Air MyAssignment %
```

Part 2, the Run from the Question and One More

```
adilshakil — -zsh — 124x31
adilshakil@ADILs-MacBook-Air MyAssignment % gcc -Wall -Wextra program.c -o program
adilshakil@ADILs-MacBook-Air MyAssignment % ./program 12 5 abc 8 12 20 3
Invalid argument ignored: abc
Valid integers in ascending order: 3 5 8 12 12 20
Minimum: 3
Maximum: 20
Average: 10.00
Median: 10.00
Second-largest distinct value: 12
adilshakil@ADILs-MacBook-Air MyAssignment % ./program 7 -3 xyz 15 7 2 99
Invalid argument ignored: xyz
Valid integers in ascending order: -3 2 7 7 15 99
Minimum: -3
Maximum: 99
Average: 21.17
Median: 7.00
Second-largest distinct value: 15
adilshakil@ADILs-MacBook-Air MyAssignment %
```

Part 2, a Decimal, Repeated Values and No Valid Input

```
adilshakil — -zsh — 124x31
adilshakil@ADILs-MacBook-Air MyAssignment % ./program 42 abc 7.5 -8 42
Invalid argument ignored: abc
Invalid argument ignored: 7.5
Valid integers in ascending order: -8 42 42
Minimum: -8
Maximum: 42
Average: 25.33
Median: 42.00
Second-largest distinct value: -8
adilshakil@ADILs-MacBook-Air MyAssignment % ./program 5 5 5
Valid integers in ascending order: 5 5 5
Minimum: 5
Maximum: 5
Average: 5.00
Median: 5.00
Second-largest distinct value: none, every value is the same
adilshakil@ADILs-MacBook-Air MyAssignment % ./program hello world
Invalid argument ignored: hello
Invalid argument ignored: world
No valid integers were given
adilshakil@ADILs-MacBook-Air MyAssignment %
```

All ten cases came out right, and the sixth one matches the output printed in the question line for line.

Part 1 held me up the longest, and in the end it was one word. I had written `va_arg(args, float)`, because weights like 0.3 looked like small numbers to me and float felt like the right size for them. The program then printed things like 0.00 and -nan, and nothing I changed in the arithmetic made any difference, which is what finally told me the problem was in the reading and not in the maths. Anything of type float that is passed into a function with "..." in it gets turned into a double before it arrives, so asking for a float back reads only half of one and whatever happens to be next to it. One word changed to double and all five calls were right at once.

The second thing in Part 1 was where to put the zero check. I had it inside the loop to begin with, next to the test for a negative weight, and call 3 still went through, because no single weight there is negative, they are just all zero. The total only becomes zero once the loop is finished, so the check belongs after it, and without it the division would have given inf.

Part 2 started with a funny one. My first run reported `./program` itself as an invalid argument, because I had begun the loop at 0 and `argv[0]` is the name of the program. Starting at 1 was the whole fix.

The part of Part 2 I had to actually think about was the second-largest distinct value. For 3 5 8 12 12 20 the answer is 12, not 20, so it is not simply the element before the last one when the largest appears twice. Walking back from `n - 2` and stopping at the first number that is different from the maximum handles that, and it also handles 5 5 5, where there is nothing to find at all. For that case main checks first whether the smallest and the largest are even different, and says there is none rather than printing something wrong.

I slipped 7.5 into one of the runs on purpose, because it is the kind of thing that could sneak past a weak check. It is a number, but it is not an integer, and `isInteger()` rejects it the moment it reaches the dot.